

Docker + Kubernetes + Helm 全栈速查笔记

日期：2026-05-11 用途：随时复习，防止遗忘

一、三者是什么，什么关系

Docker

= 造 船

（把应用打包成标准集装箱—镜像，跑在容器里）

Kubernetes

= 管 船队

（管理一堆容器的部署、扩缩容、故障恢复、网络互通）

Helm

= 管 船队调度手册

（把 K8s 的 YAML 配置打包成模板，一条命令部署）

比喻：

- **Docker**：你做了一道菜，封好装进外卖盒（镜像），谁都能拿到同样的味道。
- **Kubernetes**：你是餐厅经理，管 10 个厨师（Pod），有人请假自动补人（副本自愈），客人多了加座位（扩缩容）。
- **Helm**：你开了分店，不用重新教每个厨师，改一份"分店配置单"（values.yaml）就行——换城市换口味只改配方，不换流程。

核心区别

	Docker	Kubernetes	Helm
管什么	单个容器	一群容器（集群）	K8s 的配置文件（YAML 模板）
解决问题	环境一致性	高可用、弹性伸缩、服务发现	配置复用、版本管理、一键部署
类比	外卖盒	餐厅经理	分店配置手册
不用它会怎样	代码"在我电脑能跑"	容器挂了没人管，扩容靠手动	每个环境复制一份 YAML，改漏就炸

二、Docker 完整速查

2.1 核心概念

镜像（Image）= 打包好的只读模板（像 .exe 安装包）

容器（Container）= 镜像跑起来的实例（像运行的程序）

Dockerfile = 制作镜像的配方

Volume = 持久化数据（容器删了数据还在）

2.2 数据流

```

写代码 + Dockerfile
    ↓
docker build → 生成镜像 (Image)
    ↓
docker run → 跑起容器 (Container)
    ↓
用户访问容器端口

```

2.3 常用命令

```

# ===== 镜像操作 =====
docker images                # 列出本地镜像
docker pull nginx            # 拉取镜像
docker rmi <image_id>        # 删除镜像

# ===== 容器操作 =====
docker run -d -p 5000:5000 flask-app # 后台运行，端口映射
docker ps                    # 查看运行中的容器
docker ps -a                 # 查看所有容器（含已停止）
docker stop <container_id>    # 停止容器
docker rm <container_id>      # 删除容器
docker logs <container_id>    # 查看容器日志
docker exec -it <container_id> bash # 进入容器内部

# ===== 镜像构建 =====
docker build -t my-app:v1 .    # 根据当前目录 Dockerfile 构建

# ===== 数据卷 =====
docker volume create mydata    # 创建数据卷
docker run -v mydata:/app/data ... # 挂载数据卷

# ===== docker-compose =====
docker-compose up -d           # 后台启动全部服务
docker-compose down            # 停止并清理
docker-compose ps              # 查看服务状态

```

2.4 Dockerfile 核心指令

```

FROM python:3.11-slim        # 基础镜像
WORKDIR /app                  # 工作目录
COPY requirements.txt .       # 复制文件
RUN pip install -r requirements.txt # 安装依赖
COPY . .                      # 复制全部代码
EXPOSE 5000                   # 声明端口
CMD ["python", "app.py"]      # 启动命令（只能有一条）

```

2.5 常见坑

现象	原因	解决
容器一启动就退出	CMD 执行完就退出了	用 <code>-d</code> 或 CMD 跑前台进程
数据丢了	没挂载 Volume	<code>docker run -v host:/container</code>
端口冲突	宿主机端口被占用	换 <code>-p 5001:5000</code> 或其他端口

三、Kubernetes 完整速查

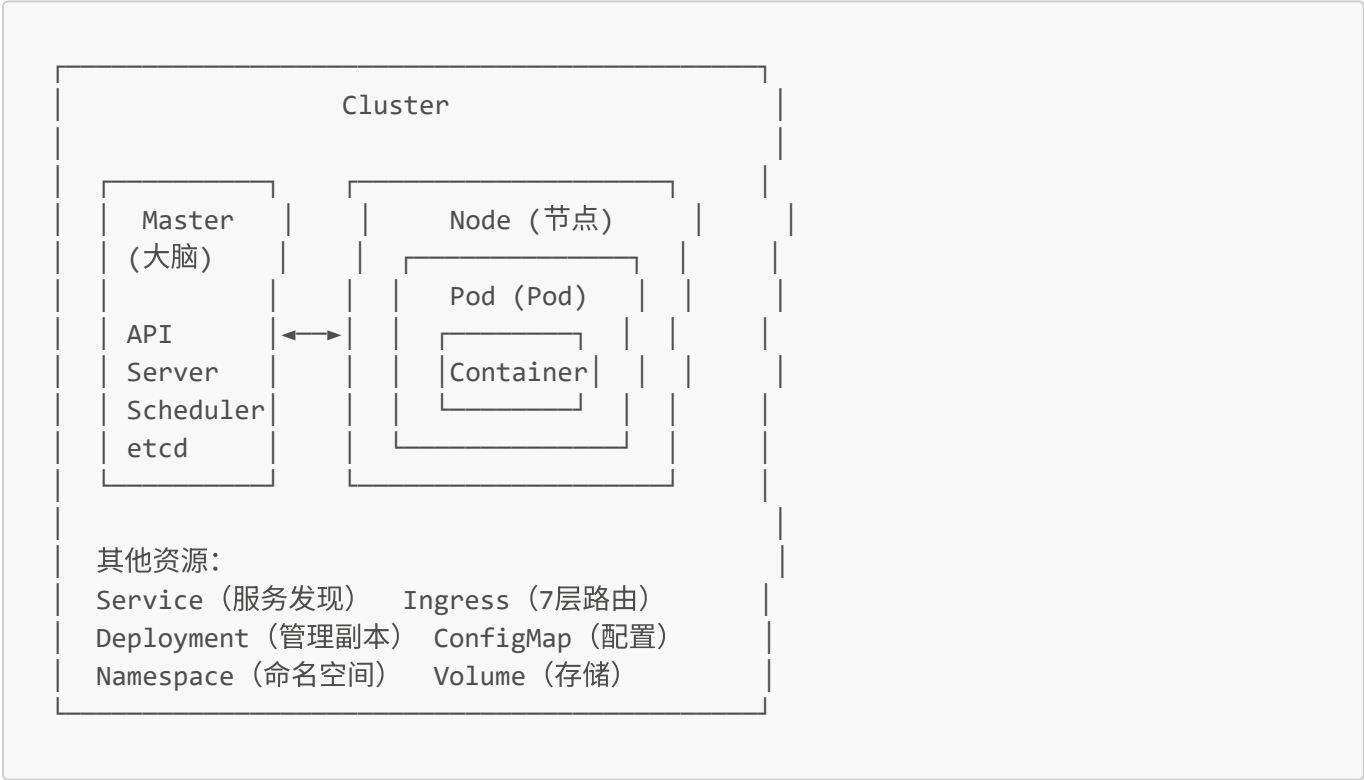
3.1 为什么需要 K8s

Docker 只管一个容器。当你有 10 个容器要同时跑、要互相通信、挂了要自动重启、要扩缩容时，手写 `docker run` 就管不过来了。

K8s 解决的问题：

- 我有 3 个 Pod，1 个挂了，怎么办？→ K8s 自动补 1 个
- 用户量翻倍了，要加实例 → 改副本数，自动扩容
- Pod 的 IP 老变，怎么稳定访问？→ Service 提供稳定入口
- 我要按路径分发请求 (/api 去 A，/ 去 B) → Ingress 7 层路由

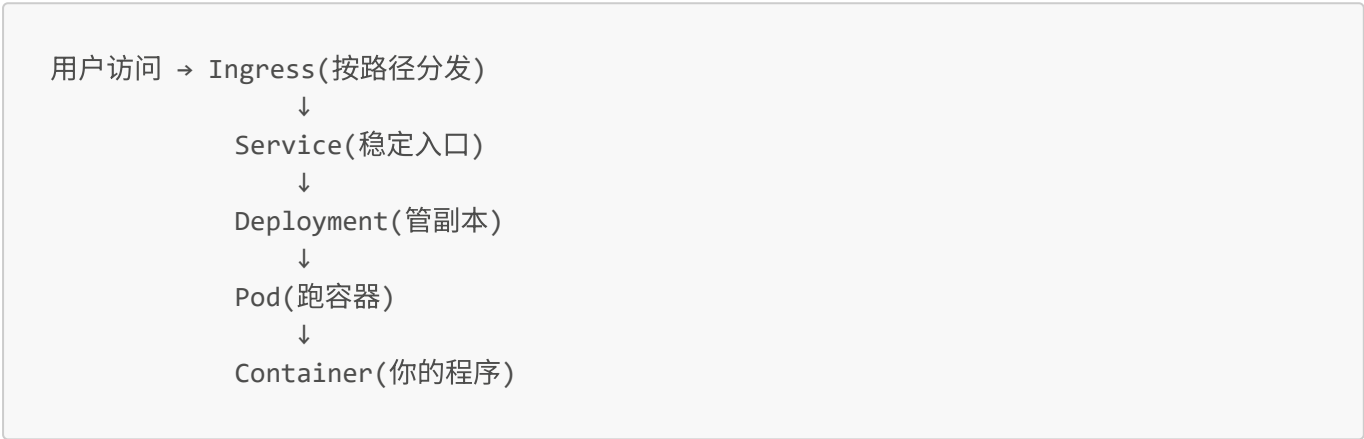
3.2 核心概念



概念	大白话	类比
Pod	K8s 最小运行单元，里面装一个或多个容器	一个工位
Deployment	管理 Pod 副本，负责"跑几个、跑什么版本"	组长，管几个人干活

概念	大白话	类比
Service	给一组 Pod 提供稳定的网络入口	公司总机，不管谁接电话
Ingress	一个端口，按路径分发到不同 Service	前台，/api 去 A 部门，/ 去 B 部门
Namespace	隔离空间，不同团队的项目互不干扰	不同办公室
ConfigMap	存配置（非敏感），像环境变量	公司规章制度
Node	一台物理/虚拟机，上面跑着 Pod	一栋办公楼

3.3 Pod ↔ Deployment ↔ Service ↔ Ingress 的关系



一个具体例子：



3.4 常用命令

```
# ===== 集群状态 =====
kubectl get nodes                # 查看节点
kubectl get pods                 # 查看 Pod
kubectl get pods -w              # 实时监控 Pod 变化
kubectl get svc                  # 查看 Service
kubectl get all                  # 查看所有资源
kubectl get pods -n <namespace> # 查看指定命名空间

# ===== 部署/更新 =====
kubectl apply -f deployment.yaml # 创建或更新
kubectl delete -f deployment.yaml # 删除
```

```

kubectl scale deployment flask-app --replicas=3 # 扩缩容

# ===== 排查问题（三板斧） =====
kubectl logs <pod_name> # 1. 看日志
kubectl describe pod <pod_name> # 2. 看事件（Events 在最后）
kubectl exec -it <pod_name> -- bash # 3. 进容器内部看

# ===== 常用快捷操作 =====
kubectl delete pod <pod_name> # 删 Pod（Deployment 会自动重建）
kubectl rollout status deployment/<name> # 查看滚动更新状态
kubectl rollout undo deployment/<name> # 回滚到上一个版本
kubectl port-forward <pod_name> 8080:80 # 本地端口转发访问

```

3.5 YAML 文件结构

```

# deployment.yaml 核心结构
apiVersion: apps/v1 # API 版本
kind: Deployment # 资源类型
metadata:
  name: flask-app # 名字
spec:
  replicas: 2 # 跑几个 Pod
  selector:
    matchLabels: # 管哪些 Pod
      app: flask-app
  template: # Pod 模板
    metadata:
      labels:
        app: flask-app # 要和 selector 匹配
    spec:
      containers:
        - name: flask # 容器名
          image: flask-app:latest # 用哪个镜像
          ports:
            - containerPort: 5000 # 容器端口

```

3.6 常见状态和排查

Pod 状态	含义	排查方法
Running	正常	-
Pending	调度中（资源不够/镜像拉不到）	<code>kubectl describe pod</code> 看 Events
ImagePullBackOff	镜像拉取失败（被墙/镜像名错）	检查镜像名、网络、用本地镜像
CrashLoopBackOff	容器反复崩溃重启	<code>kubectl logs <pod></code> 看报错
OOMKilled	内存超限被杀	调大 <code>resources.limits.memory</code>
Error	容器退出码非 0	<code>kubectl logs</code> + <code>kubectl describe</code>

3.7 k3s 特有命令

k3s 是轻量版 K8s, kubeconfig 在 `/etc/rancher/k3s/k3s.yaml`:

```
sudo k3s kubectl get pods          # 完整命令
# 或复制 kubeconfig 后直接用 kubectl:
cp /etc/rancher/k3s/k3s.yaml ~/.kube/config
kubectl get pods                    # 简化命令

sudo k3s ctr images ls              # 查看 k3s 里的镜像
```

3.8 GPU 资源管理（联旌核心场景）

```
# GPU 资源声明
resources:
  limits:
    nvidia.com/gpu: 1    # 要 1 张 GPU
```

共享方案	隔离级别	适用场景
MPS 时间片	无隔离	所有 GPU, 轻量
vGPU 虚拟化	软件隔离	需要许可证
MIG 切分	硬件隔离	A100/H100 数据中心

排查: `nvidia-smi` → `kubectl describe pod` → `kubectl exec -- nvidia-smi`

四、Helm 完整速查

4.1 为什么需要 Helm

问题: 一个应用要写 5 个 YAML (Deployment、Service、Ingress...), 每次部署敲 5 次 `kubectl apply`。换环境 (开发→测试→生产) 每个 YAML 都要改端口和镜像。

Helm 的答案: 把 YAML 写成模板, 变量抽到 `values.yaml`, 一条命令 `helm install` 搞定。

4.2 核心概念

```
Chart (图表/包) = 一个文件夹 = K8s 的"安装包"
├─ Chart.yaml      # 包的身份证 (名称、版本)
├─ values.yaml     # 配置文件 (所有变量)
├─ .helmignore     # 忽略名单
└─ templates/      # YAML 模板 (用 {{ .Values.xxx }} 引用变量)
```

Release (发布实例) = 用 `helm install` 装一个 Chart 后跑起来的东西
同一个 Chart 可以装多次, 每次产生一个独立的 Release

values (变量) = 配置差异点
values.yaml 里定义默认值, 安装时可用 --set 覆盖

4.3 Chart 结构详解

```
my-chart/
├── Chart.yaml           # 元信息
│   ├── name: my-chart  # 包名
│   ├── version: 0.1.0  # Chart 版本 (语义化版本)
│   └── appVersion: "1.0" # 应用版本
├── values.yaml          # 默认配置
│   ├── replicaCount: 1
│   ├── image:
│   │   ├── repository: nginx
│   │   └── tag: "latest"
│   └── service:
│       ├── type: ClusterIP
│       └── port: 80
└── templates/
    ├── deployment.yaml  # 模板: Deployment YAML + {{ .Values.xxx }}
    ├── service.yaml     # 模板: Service YAML + {{ .Values.xxx }}
    ├── _helpers.tpl     # 公共函数 (fullname、labels)
    ├── NOTES.txt        # 安装后显示的说明
    └── tests/
        └── test-connection.yaml # 安装后的测试
```

4.4 Go 模板语法 (三种就够)

```
# 1. 取值: 读 values.yaml 里的变量
{{ .Values.replicaCount }}
{{ .Values.image.repository }}

# 2. 条件判断
{{- if .Values.ingress.enabled }}
... 这里的内容只在 enabled 为 true 时输出 ...
{{- end }}

# 3. 调用公共函数 (定义在 _helpers.tpl)
{{ include "my-chart.fullname" . }}
```

- 的作用: {{- 和 -}} 去掉模板前后的空行, 保持生成的 YAML 整洁。

4.5 常用命令

```
# ===== 安装/卸载 =====
helm install <release_name> ./my-chart # 安装 Chart
```

```

helm install <name> . --dry-run --debug          # 模拟安装（只看生成的 YAML，不部署）
helm uninstall <release_name>                   # 卸载

# ===== 升级/回滚 =====
helm upgrade <release_name> ./my-chart          # 升级
helm upgrade <name> . --set replicaCount=3      # 升级并改副本数
helm upgrade <name> . --set image.tag=v2 --reuse-values # 只改一个值，其他不变
helm rollback <release_name> <revision>        # 回滚到指定版本
helm history <release_name>                    # 查看版本历史

# ===== 查看 =====
helm list                                       # 列出已安装的 Release
helm status <release_name>                    # 查看某个 Release 的状态
helm template <release_name> ./my-chart       # 渲染模板（生成最终 YAML，不部署）
helm template <name> . > rendered.yaml        # 保存到文件

```

4.6 --set vs --reuse-values

```

# 场景：之前安装时设了镜像地址，现在只想改副本数

# 不用 --reuse-values（危险！其他值会恢复默认）
helm upgrade my-release . --set replicaCount=3
# 结果：镜像地址恢复成 Chart 默认值，可能拉不到镜像

# 用 --reuse-values（安全，只改指定的值）
helm upgrade my-release . --set replicaCount=3 --reuse-values
# 结果：镜像地址不变，只有副本数变成 3

```

4.7 命名规则

Helm 自动生成的资源名格式：<release_name>-<chart_name>

```

helm install flask-release ./helm-flask
↓
资源名：flask-release-helm-flask
Pod：flask-release-helm-flask-7677b54c4c-lsvj6
Service：flask-release-helm-flask

```

4.8 Helm 生命周期

```

helm install    → REVISION 1 deployed
helm upgrade    → REVISION 2 deployed, REVISION 1 → superseded
helm upgrade    → REVISION 3 deployed, REVISION 2 → superseded

```


helm rollback 1 → REVISION 4 deployed, 内容是 REVISION 1 的配置
helm uninstall → 全部删除, history 保留 (可加 --keep-history 保留)

五、三者串联：一个完整部署流程

```
# Step 1: 写好代码
app.py

# Step 2: 用 Docker 打包
docker build -t flask-app:latest .

# Step 3: 用 Helm 管理部署
helm create helm-flask           # 生成 Chart
vim values.yaml                  # 改镜像、端口、副本数
helm install flask-release ./helm-flask  # 部署到 K8s

# Step 4: 日常运维
helm upgrade flask-release . --set replicaCount=5  # 扩容
helm rollback flask-release 1                      # 出问题回滚
helm history flask-release                          # 看历史
```

六、命令对照表

动作	Docker	Kubernetes	Helm
列出运行中的	<code>docker ps</code>	<code>kubectl get pods</code>	<code>helm list</code>
启动/安装	<code>docker run</code>	<code>kubectl apply -f</code>	<code>helm install</code>
停止/卸载	<code>docker stop</code>	<code>kubectl delete -f</code>	<code>helm uninstall</code>
查看日志	<code>docker logs</code>	<code>kubectl logs</code>	<code>helm status</code>
进入	<code>docker exec</code>	<code>kubectl exec</code>	-
更新配置	重建容器	<code>kubectl apply</code> + 改 YAML	<code>helm upgrade</code>
回滚	重新 run 旧镜像	<code>kubectl rollout undo</code>	<code>helm rollback</code>
查看历史	-	<code>kubectl rollout history</code>	<code>helm history</code>

七、常见问题排查流程

7.1 容器起不来

1. `docker ps -a`

2. `docker logs <id>`
- 容器在吗？状态是什么？

→ 应用报错了什么？

3. 看 Dockerfile → CMD 对吗? 端口对吗?

7.2 Pod 起不来

1. `kubectl get pods` → Pod 状态是什么?
2. `kubectl describe pod` → Events 里有什么错误?
 - `ImagePullBackOff` → 镜像名/网络问题
 - `CrashLoopBackOff` → 应用崩溃, `kubectl logs` 看
 - `Pending` → 资源不够/调度不了
3. `kubectl logs <pod>` → 应用日志
4. `kubectl exec -- bash` → 进容器看环境

7.3 Helm 部署失败

1. `helm install --dry-run --debug` → 模板渲染对不对?
2. `helm template .` → 生成的 YAML 对不对?
3. 检查 `values.yaml` → 镜像名/端口/副本写对了吗?
4. `kubectl get pods` → Pod 创建了吗? 什么状态?

八、核心原则（背下来）

1. **Docker 管环境**: 保证"开发、测试、生产跑的一样"
2. **K8s 管规模**: 保证"挂了自动补, 不够自动加"
3. **Helm 管配置**: 保证"换环境不改模板, 只改变量"
4. **排查三板斧**: 看状态 → 看事件/日志 → 进容器
5. **K8s 最小单位是 Pod, 不是容器**
6. **Service 是稳定的网络入口, Pod 的 IP 会变但 Service 的不会**
7. **Helm 的 rollback 是创建新版本, 不是删除旧版本**
8. **values.yaml 是唯一的"需要改的地方", 别去改 templates/**